

ELENE 6908 Embedded AI Final Project Report

Memory Recover

Yonghao Lin (yl5763), Zhenqi Li (zl3508), Yang Cao (yc4535)

1 Problem Statement

Our system is designed for individuals concerned about losing meaningful moments or memories, with the hope of retrieving and reliving them. These users often face three core challenges: the first one is memory decline, which refers to difficulty recalling past experiences or where personal items were placed; the second one is forgotten items, as people often lose track of where things were put; and the last one is privacy concerns, since many existing memory-assist systems upload data to the cloud, risking the exposure of sensitive images or audio.

To address these issues, we have developed a lightweight system that operates entirely on-device to preserve user privacy. It listens for **keywords**, analyzes both **images and text**, and leverages **vision language models (VLMs)** and **small language models (SLMs)** using a **retrieval-augmented generation (RAG)** framework. By combining these components, the system helps users securely and efficiently recall past events and locate lost items. For more details, please explore our project resources.¹²

2 System Design

Our system consists of three main models: keyword spotting, taking photos, and asking questions. The architecture of the system is shown in Figure 1.

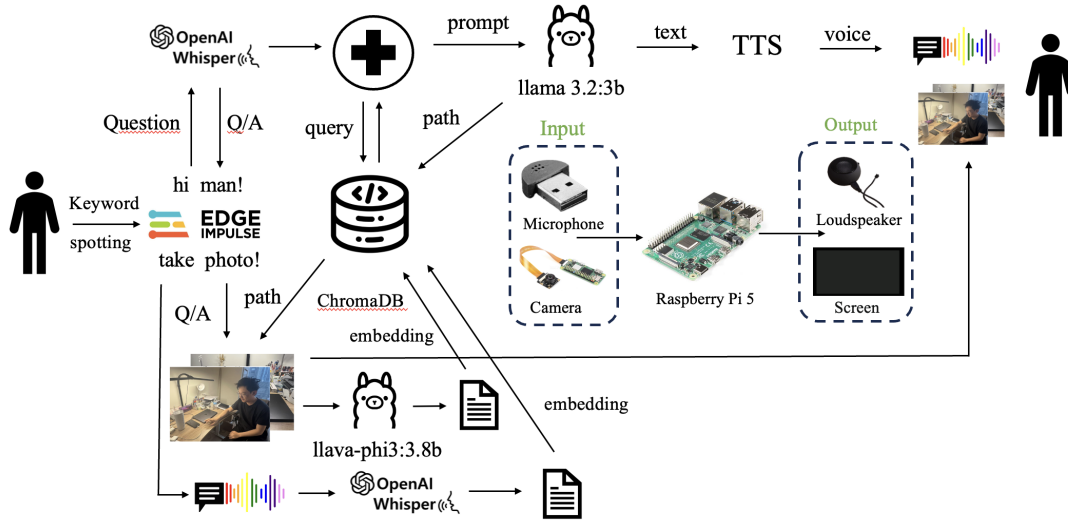


Figure 1: The architecture of the memory recovery system.

The first component is keyword spotting. To enable efficient and hands-free interaction, we trained a lightweight four-class audio classification model using Edge Impulse. This model is capable of detecting predefined phrases, namely “hi man” and “take photo,” while filtering out unrelated speech (“unknown”) and environmental noise (“noise”). Only when one of the valid keywords is recognized does the system activate the subsequent processing steps. A secondary confirmation part is implemented to reduce false triggers; Users can respond with ‘yes’ or ‘no’ to confirm commands, also detected by keyword spotting.

¹GitHub Repository: <https://github.com/zl3508/Memory-Recover>

²Demo Video: <https://www.youtube.com/watch?v=puY44id-oYs>

The second model is taking photos. Upon detecting the command ‘take photo’, the system captures an image using the onboard camera of the Raspberry Pi. The user is then asked to verbally describe the captured moment and their speech is transcribed into text using OpenAI Whisper. Optionally, the system can invoke a vision language model (llava-phi3:3.8b) to generate an automatic description of the image. User-generated and model-generated descriptions are stored in JSON format, along with metadata such as timestamps and file paths. These entries are then embedded into a local vector database (ChromaDB), allowing future retrieval and reasoning without relying on cloud services.

The third component handles question answering. When the system detects the keyword “hi man”, it interprets the user’s speech as a memory-related query (e.g., “Where did I last see my phone?”). The spoken question is first transcribed, then converted into a vector embedding and used to perform semantic retrieval over the locally stored memory database (ChromaDB). The system retrieves the top-k most relevant entries, considering both user-provided notes and image descriptions generated by the VLM (llava-phi3:3.8b). These retrieved memories, along with the original question, are passed to a SLM (llama3.2:3b), which reasons over the combined context using a structured prompt. The model generates a natural-language response, and if applicable, returns associated image paths. Finally, the system displays the image(s) on screen and reads the answer aloud using text-to-speech (TTS). This multimodal interaction enables rich, context-aware recall of meaningful past moments.

3 Experimental Results

To demonstrate the functionality and effectiveness of our memory assistant system, we conducted a complete experimental cycle from memory recording to retrieval. This includes real-time voice interaction, multimodal memory storage, semantic search, and vision-language reasoning. All components are executed locally on a Raspberry Pi to ensure low-latency and data privacy.

To reduce system latency and ensure responsiveness, we implemented keyword spotting models to detect core activation phrases. Specifically, the phrases “hi man” and ‘take photo’ are used to wake up the assistant and record a moment, while “yes” and “no” are used for user confirmation. We trained two separate 4-class classifiers to handle these tasks shown in Figure 2.

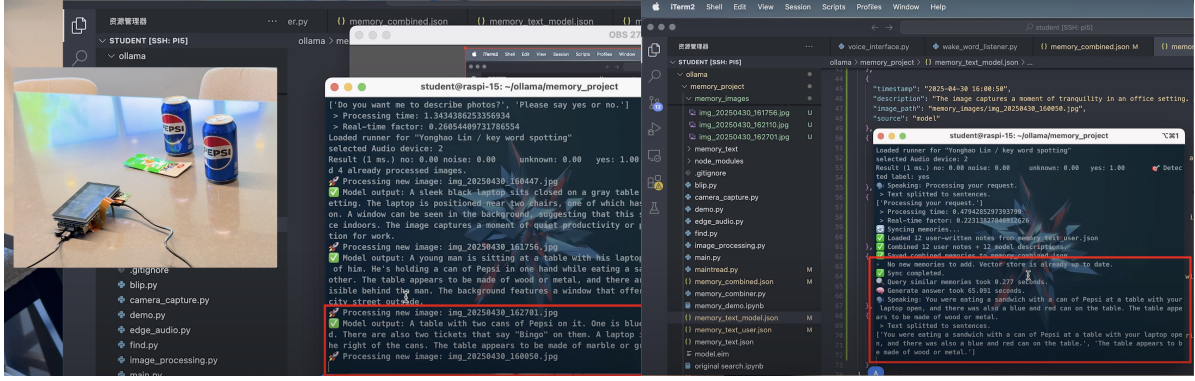


Figure 2: Confusion matrices of keyword spotting models.

After a successful voice command such as “take photo”, the system captures an image, records user’s description and optionally (if user needs) processes it using VLM. In our experiment, the user provided a vague description: “These are my food.” Despite the limited input, the VLM generated a much more informative output: a table with two cans of Pepsi on it—one blue and one red. It also identified what it described as two tickets labeled “Bingo” and noted a laptop positioned to the right of the cans. The result is shown in Figure 3(a). It illustrates both the strengths and limitations of VLMs. On the positive side, the model successfully detected the presence of two Pepsi cans. However, it also misinterpreted the colors

of the Pepsi logo as two different types of drinks and mistakenly classified a Chinese snack as “tickets.” To address such issues, our system combines user descriptions with VLM-generated outputs to improve robustness and accuracy.

We then tested the system’s memory retrieval function by asking, “Where is my Pepsi? I forgot.” The spoken query was transcribed and embedded, then used to perform semantic search over the stored memory vector DB. The top relevant entries were retrieved and passed to the LLaMA 3.2 model and returned a relevant response. The output is shown in Figure 3(b).



((a)) VLM output for the image.

((b)) System-generated response to the query.

Figure 3: The results of VLM output and system-generated response.

To support its answer, the system returned two reference images: one shows the user eating with a Pepsi can next to a laptop, and the other displays two Pepsi cans and the previously misidentified snack. The result is shown in Figure 4.



((a)) Reference Image 1/2 — user with Pepsi and sandwich.

((b)) Reference Image 2/2 — two Pepsi cans and snack.

Figure 4: Images retrieved by the system in response to the user’s query.

Our system achieves an average response time between 40 to 80 seconds, depending on the complexity of the query and the reasoning time required by the language model. Most of the delay is concentrated in the SLM reasoning and generation step, while keyword spotting and retrieval are nearly instantaneous.

4 Conclusion

In summary, our system demonstrates the feasibility of a lightweight, privacy-preserving memory assistant that integrates speech, vision, and reasoning on-device. In the future, we plan to explore using CLIP to directly embed images into the same vector space as text, enabling faster response time.